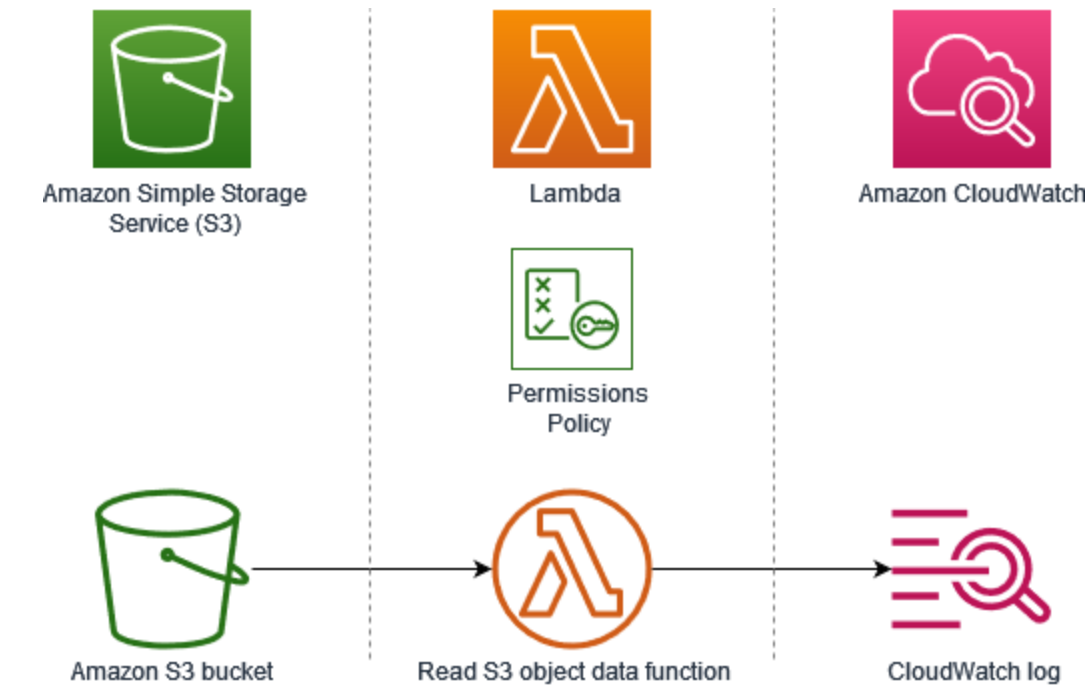
Tutorial: Using an Amazon S3 trigger to invoke a Lambda function

In this tutorial, you use the console to create a Lambda function and configure a trigger for an Amazon Simple Storage Service (Amazon S3) bucket. Every time that you add an object to your Amazon S3 bucket, your function runs and outputs the object type to Amazon CloudWatch Logs.

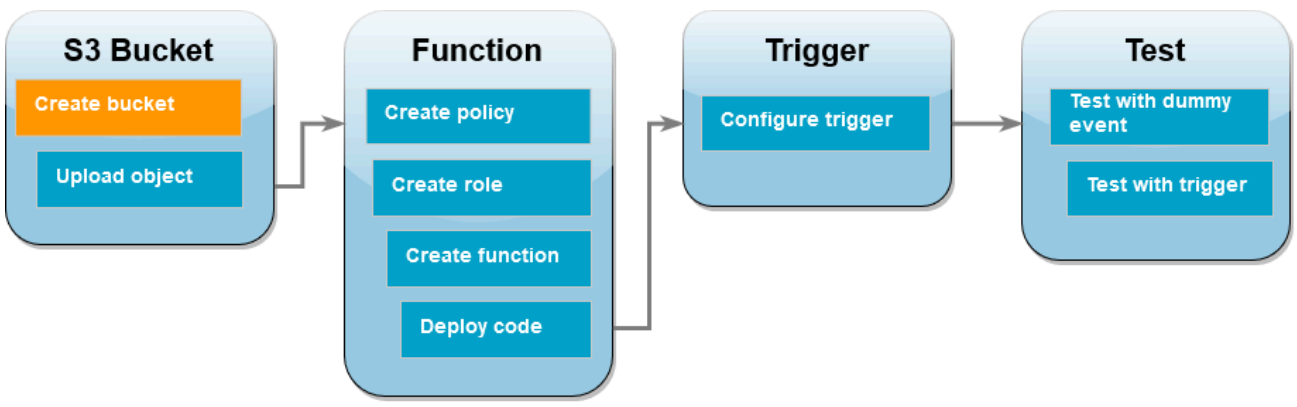


This tutorial demonstrates how to:

1. Create an Amazon S3 bucket.
2. Create a Lambda function that returns the object type of objects in an Amazon S3 bucket.
3. Configure a Lambda trigger that invokes your function when objects are uploaded to your bucket.
4. Test your function, first with a dummy event, and then using the trigger.

By completing these steps, you'll learn how to configure a Lambda function to run whenever objects are added to or deleted from an Amazon S3 bucket. You can complete this tutorial using only the AWS Management Console.

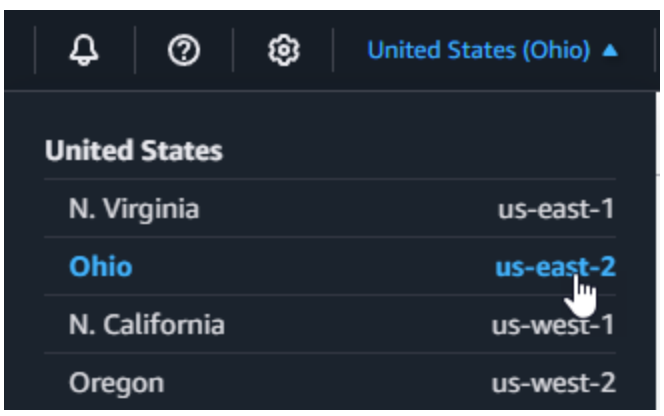
Create an Amazon S3 bucket



To create an Amazon S3 bucket

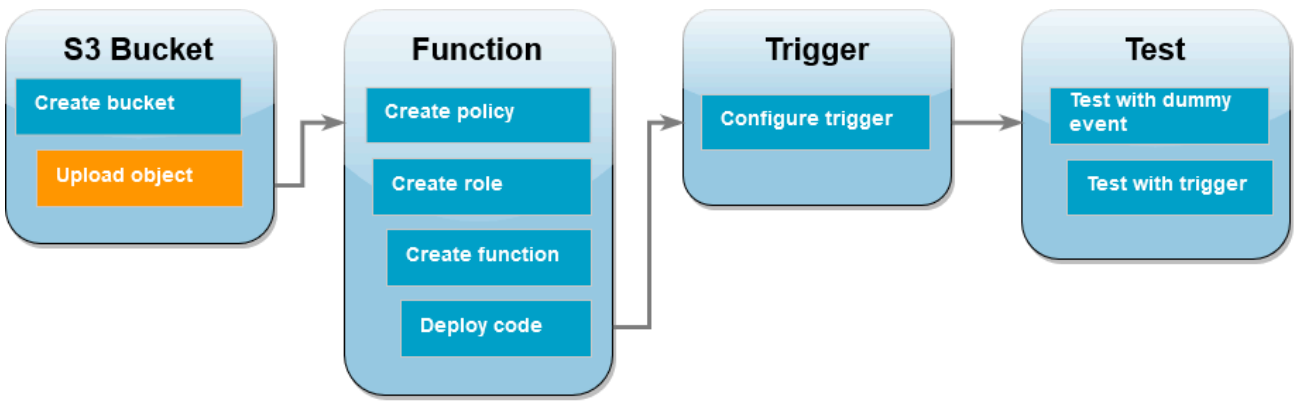
1. Open the Amazon S3 console and select the **General purpose buckets** page.

2. Select the AWS Region closest to your geographical location. You can change your region using the drop-down list at the top of the screen. Later in the tutorial, you must create your Lambda function in the same Region.



3. Choose **Create bucket**.
4. Under **General configuration**, do the following:
   a. For **Bucket type**, ensure **General purpose** is selected.
   b. For **Bucket name**, enter a globally unique name that meets the Amazon S3 Bucket naming rules. Bucket names can contain only lower case letters, numbers, dots (.), and hyphens (-).
5. Leave all other options set to their default values and choose **Create bucket**.

Upload a test object to your bucket

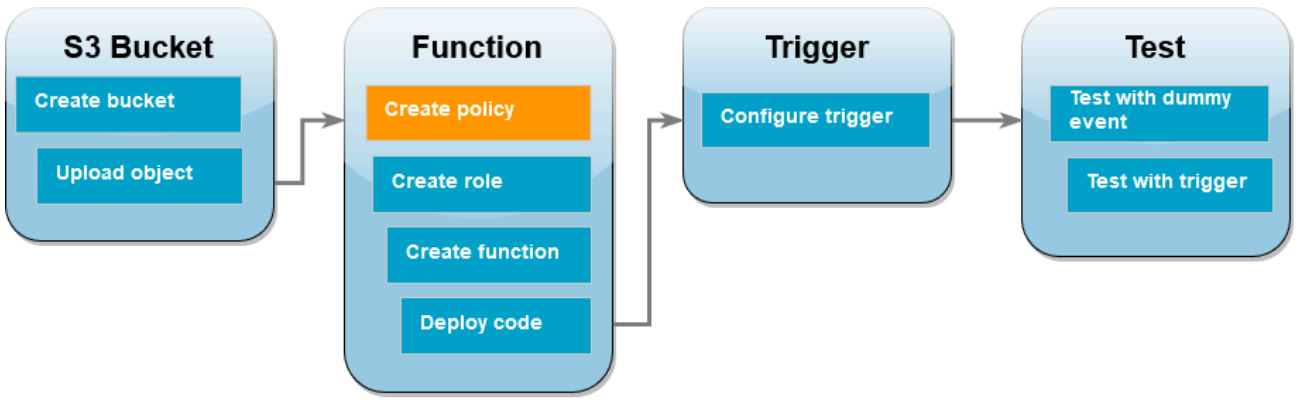


To upload a test object

1. Open the Buckets page of the Amazon S3 console and choose the bucket you created during the previous step.
2. Choose **Upload**.
3. Choose **Add files** and select the object that you want to upload. You can select any file (for example, `HappyFace.jpg`).
4. Choose **Open**, then choose **Upload**.

Later in the tutorial, you'll test your Lambda function using this object.

Create a permissions policy



Create a permissions policy that allows Lambda to get objects from an Amazon S3 bucket and to write to Amazon CloudWatch Logs.

To create the policy

1. Open the Policies page of the IAM console.
2. Choose **Create Policy**.
3. Choose the **JSON** tab, and then paste the following custom policy into the JSON editor.

JSON

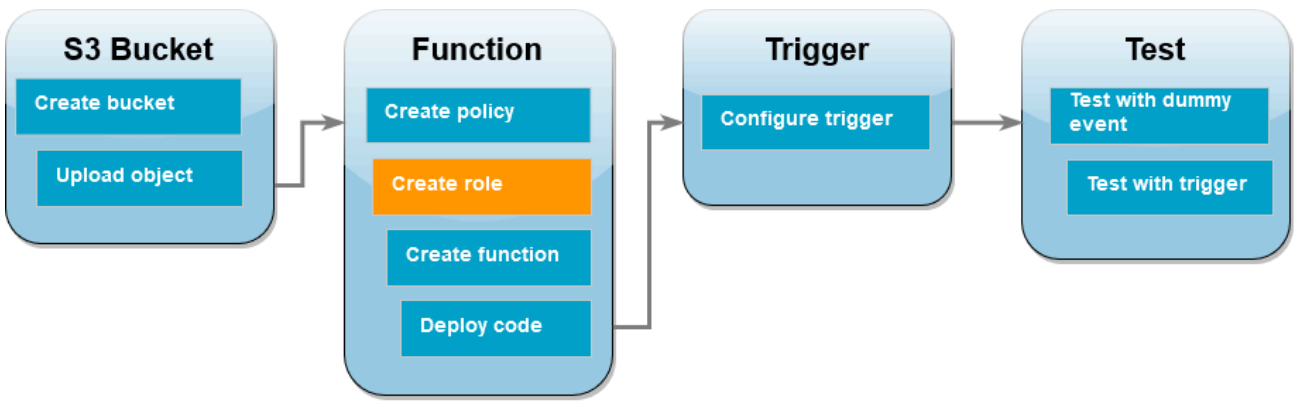
```
{
    "Version":"2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "logs:PutLogEvents",
                "logs:CreateLogGroup",
                "logs:CreateLogStream"
            ],
            "Resource": "arn:aws:logs:*:*:*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:GetObject"
            ],
            "Resource": "arn:aws:s3:::*/*"
        }
    ]
}
```

Provide feedback

4. Choose **Next: Tags**.
5. Choose **Next: Review**.

6. Under **Review policy**, for the policy **Name**, enter `s3-trigger-tutorial`.
7. Choose **Create policy**.

Create an execution role

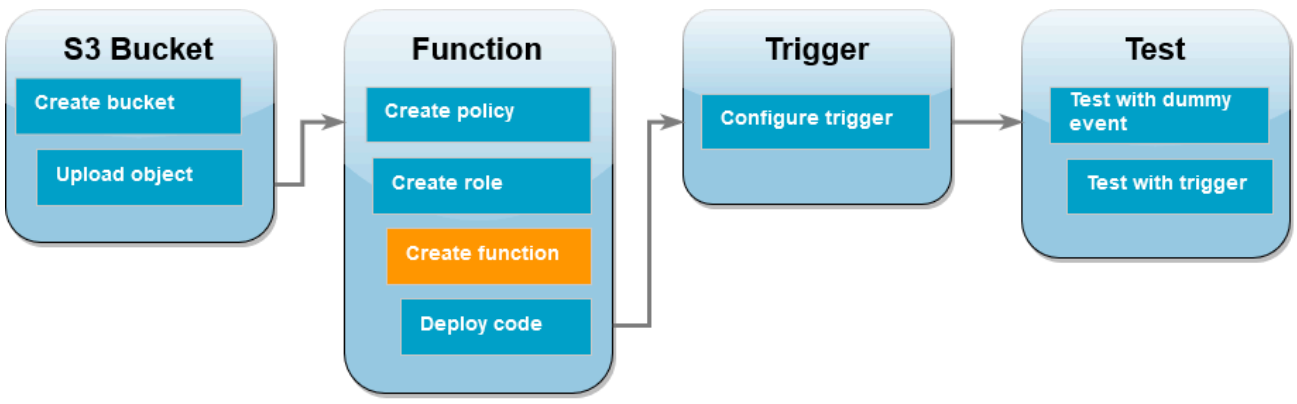


An execution roleis an AWS Identity and Access Management (IAM) role that grants a Lambda function permission to access AWS services and resources. In this step, create an execution role using the permissions policy that you created in the previous step.

To create an execution role and attach your custom permissions policy

1. Open the Roles page of the IAM console.
2. Choose **Create role**.
3. For the type of trusted entity, choose **AWS service**, then for the use case, choose **Lambda**.
4. Choose **Next**.
5. In the policy search box, enter `s3-trigger-tutorial`.
6. In the search results, select the policy that you created (`s3-trigger-tutorial`), and then choose **Next**.
7. Under **Role details**, for the **Role name**, enter `lambda-s3-trigger-role`, then choose **Create role**.

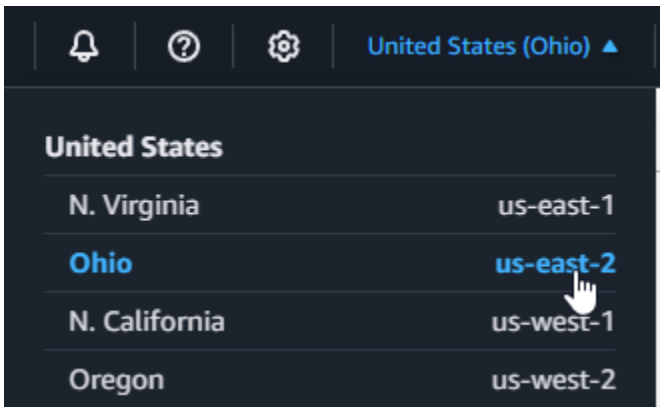
Create the Lambda function



Create a Lambda function in the console using the Python 3.14 runtime.

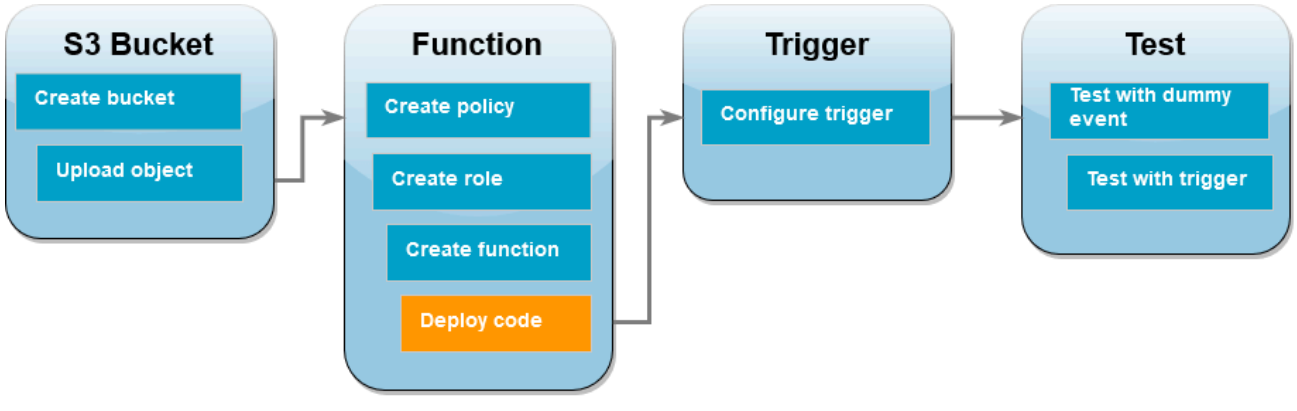
To create the Lambda function

1. Open the Functions page of the Lambda console.
2. Make sure you're working in the same AWS Region you created your Amazon S3 bucket in. You can change your Region using the drop-down list at the top of the screen.



3. Choose **Create function**.
4. Choose **Author from scratch**
5. Under **Basic information**, do the following:
   a. For **Function name**, enter `s3-trigger-tutorial`
   b. For **Runtime**, choose **Python 3.14**.
   c. For **Architecture**, choose **x86_64**.
6. In the **Change default execution role** tab, do the following:
   a. Expand the tab, then choose **Use an existing role**.
   b. Select the `lambda-s3-trigger-role` you created earlier.
7. Choose **Create function**.

Deploy the function code



This tutorial uses the Python 3.14 runtime, but we've also provided example code files for other runtimes. You can select the tab in the following box to see the code for the runtime you're interested in.

The Lambda function retrieves the key name of the uploaded object and the name of the bucket from the `event` parameter it receives from Amazon S3. The function then uses the get_object method from the AWS SDK for Python (Boto3) to retrieve the object's metadata, including the content type (MIME type) of the uploaded object.

To deploy the function code

1. Choose the **Python** tab in the following box and copy the code.

.NET | Go | Java | JavaScript | PH

SDK for JavaScript (v3)

Note

There's more on GitHub. Find the complete example and learn how to set up and run in

the Serverless examples repository.

Consuming an S3 event with Lambda using JavaScript.

```
import { S3Client,
HeadObjectCommand } from
"@aws-sdk/client-s3";

const client = new
S3Client();

export const handler =
async (event, context) =>
{

    // Get the object from
the event and show its
content type
    const bucket =
event.Records[0].s3.bucket
.name;
    const key =
decodeURIComponent(event.R
ecords[0].s3.object.key.re
place(/\+/g, ' '));

    try {
        const {
ContentType } = await
client.send(new
HeadObjectCommand({
            Bucket:
bucket,
            Key: key,
        }));


console.log('CONTENT
TYPE:', ContentType);
        return
ContentType;

    } catch (err) {
        console.log(err);
        const message =
`Error getting object
${key} from bucket
${bucket}. Make sure they
exist and your bucket is
in the same region as this
function.`;

console.log(message);
        throw new
Error(message);
    }
};
```

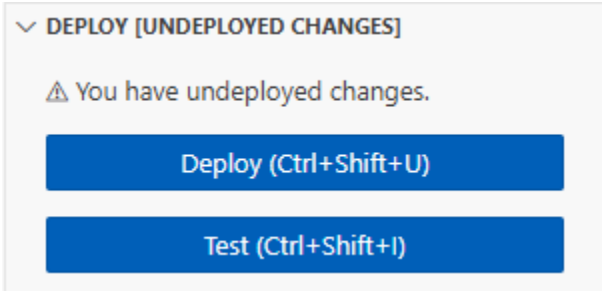
Provide feedback

Consuming an S3 event with Lambda using TypeScript.

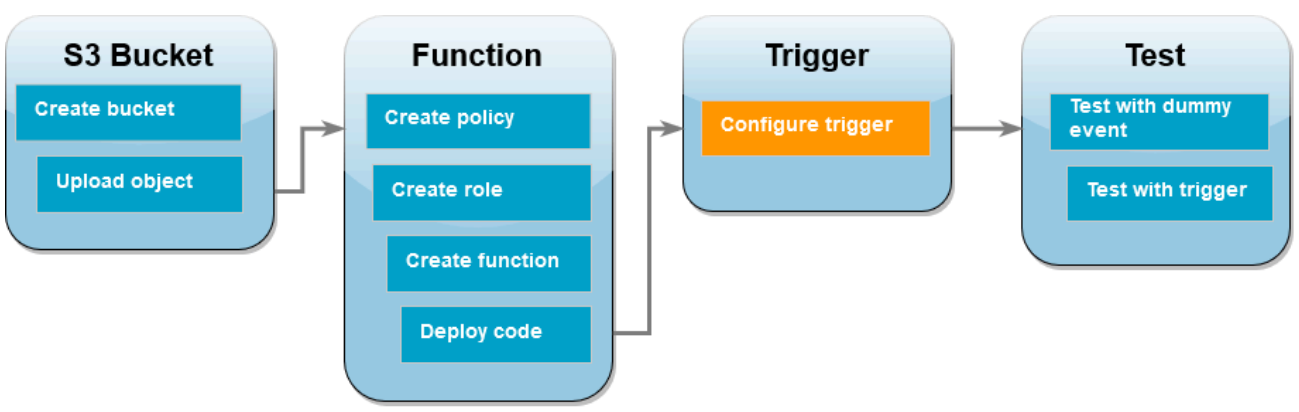
```
// Copyright Amazon.com, Inc. or its affiliates. All Rights Reserved.
// SPDX-License-Identifier: Apache-2.0
import { S3Event } from 'aws-lambda';
import { S3Client, HeadObjectCommand } from '@aws-sdk/client-s3';

const s3 = new S3Client({ region: process.env.AWS_REGION });

export const handler = async (event: S3Event): Promise<string | undefined> => {
  // Get the object from the event and show its content type
  const bucket = event.Records[0].s3.bucket.name;
  const key = decodeURIComponent(event.Records[0].s3.object.key.replace(/\+/g, ' '));
  const params = {
    Bucket: bucket,
    Key: key,
  };
  try {
    const { ContentType } = await s3.send(new HeadObjectCommand(params));
    console.log('CONTENT TYPE:', ContentType);
    return ContentType;
  } catch (err) {
    console.log(err);
    const message = `Error getting object ${key} from bucket ${bucket}. Make sure they exist and your bucket is in the same region as this function.`;
    console.log(message);
    throw new Error(message);
  }
};
```

Provide feedback

2. In the **Code source** pane on the Lambda console, paste the code into the code editor, replacing the code that Lambda created.
3. In the **DEPLOY** section, choose **Deploy** to update your function's code:

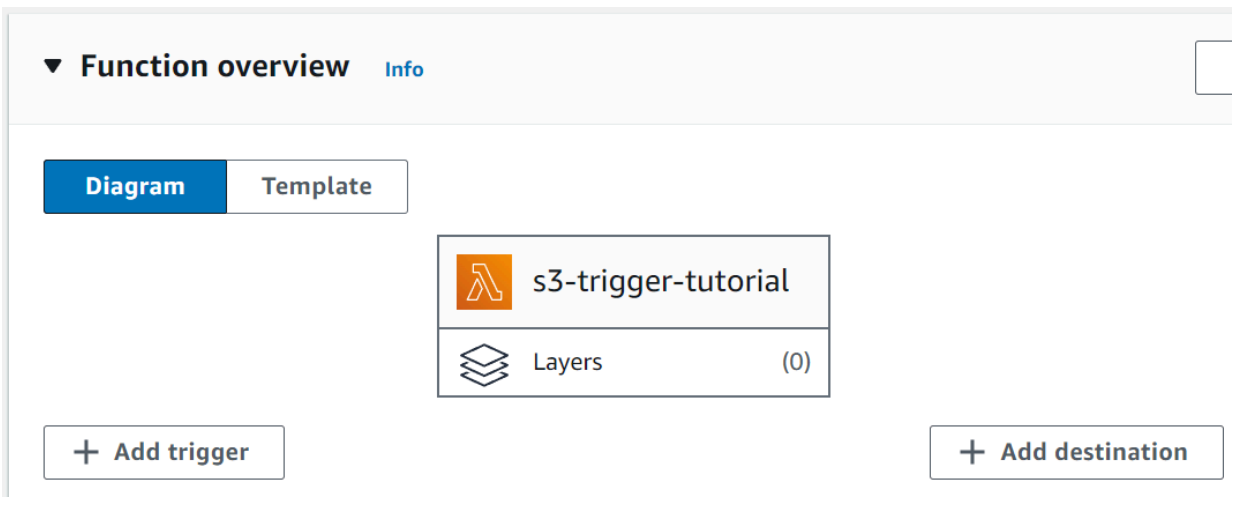


Create the Amazon S3 trigger



To create the Amazon S3 trigger

1. In the **Function overview** pane, choose **Add trigger**.



2. Select **S3**.
3. Under **Bucket**, select the bucket you created earlier in the tutorial.
4. Under **Event types**, be sure that **All object create events** is selected.
5. Under **Recursive invocation**, select the check box to acknowledge that using the same Amazon S3 bucket for input and output is not recommended.
6. Choose **Add**.

> **Note**
>
> When you create an Amazon S3 trigger for a Lambda function using the Lambda console, Amazon S3 configures an event notificationon the bucket you specify. Before configuring this event notification, Amazon S3 performs a series of checks to

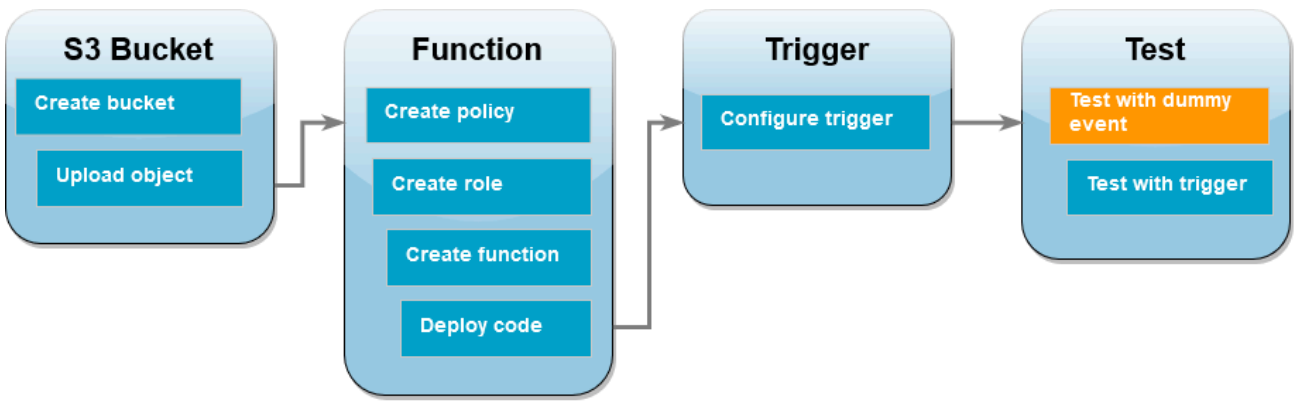
confirm that the event destination exists and has the required IAM policies. Amazon S3 also performs these tests on any other event notifications configured for that bucket.

Because of this check, if the bucket has previously configured event destinations for resources that no longer exist, or for resources that don't have the required permissions policies, Amazon S3 won't be able to create the new event notification. You'll see the following error message indicating that your trigger couldn't be created:

```
An error occurred when creating the trigger: Unable to
validate the following destination configurations.
```

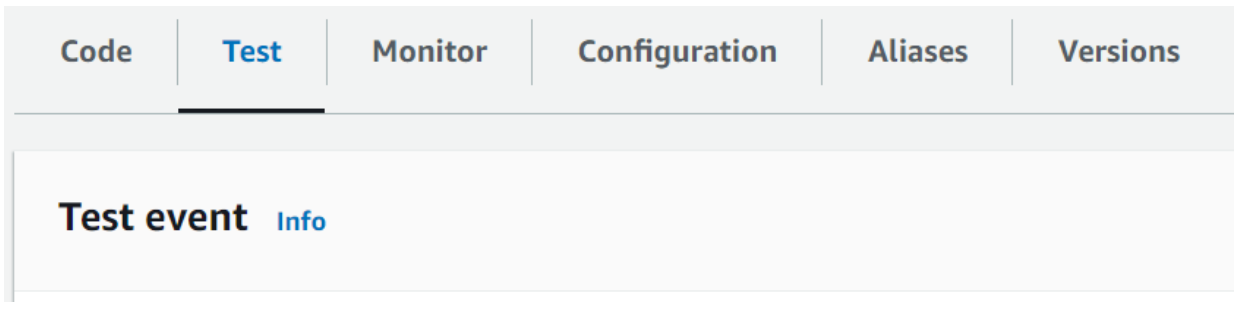
You can see this error if you previously configured a trigger for another Lambda function using the same bucket, and you have since deleted the function or modified its permissions policies.

Test your Lambda function with a dummy event



To test the Lambda function with a dummy event

1. In the Lambda console page for your function, choose the **Test** tab.



2. For **Event name**, enter `MyTestEvent`.
3. In the **Event JSON**, paste the following test event. Be sure to replace these values:
   - Replace `us-east-1` with the region you created your Amazon S3 bucket in.
   - Replace both instances of `amzn-s3-demo-bucket` with the name of your own Amazon S3 bucket.
   - Replace `test%2FKey` with the name of the test object you uploaded to your bucket earlier (for example, `HappyFace.jpg`).

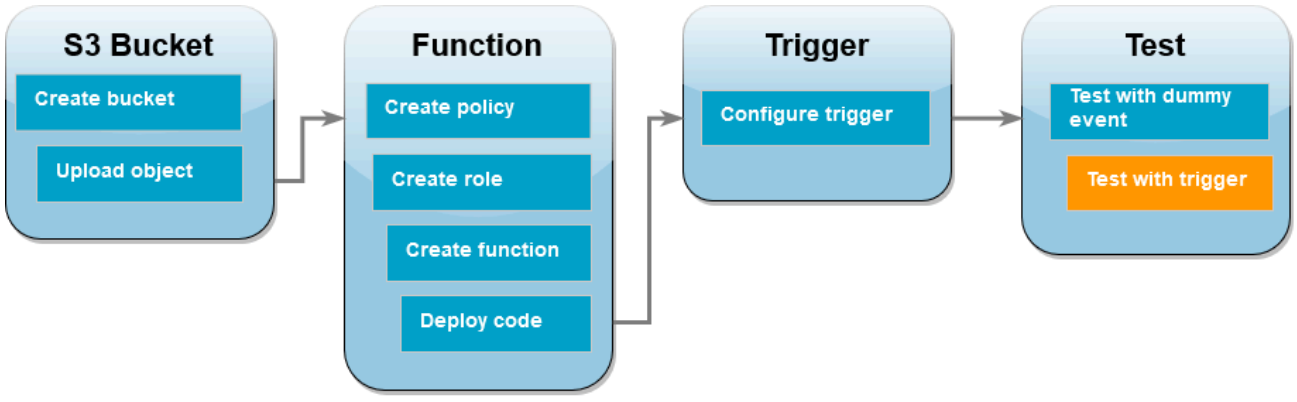
```
{
  "Records": [
    {
      "eventVersion": "2.0",
      "eventSource": "aws:s3",
      "awsRegion": "us-east-1",
      "eventTime": "1970-01-01T00:00:00.000Z",
```

```
      "eventName": "ObjectCreated:Put",
      "userIdentity": {
        "principalId": "EXAMPLE"
      },
      "requestParameters": {
        "sourceIPAddress": "127.0.0.1"
      },
      "responseElements": {
        "x-amz-request-id": "EXAMPLE123456789",
        "x-amz-id-2":
"EXAMPLE123/5678abcdefghijklambdaisawesome/mnopqrstuvwxyzABCDEFGH
"
      },
      "s3": {
        "s3SchemaVersion": "1.0",
        "configurationId": "testConfigRule",
        "bucket": {
          "name": "amzn-s3-demo-bucket",
          "ownerIdentity": {
            "principalId": "EXAMPLE"
          },
          "arn": "arn:aws:s3:::amzn-s3-demo-bucket"
        },
        "object": {
          "key": "test%2Fkey",
          "size": 1024,
          "eTag": "0123456789abcdef0123456789abcdef",
          "sequencer": "0A1B2C3D4E5F678901"
        }
      }
    }
  ]
}
```

4. Choose **Save**.
5. Choose **Test**.
6. If your function runs successfully, you'll see output similar to the following in the **Execution results** tab.

```
Response
"image/jpeg"

Function Logs
START RequestId: 12b3cae7-5f4e-415e-93e6-416b8f8b66e6 Version:
$LATEST
2021-02-18T21:40:59.280Z    12b3cae7-5f4e-415e-93e6-416b8f8b66e6
INFO    INPUT BUCKET AND KEY:  { Bucket: 'amzn-s3-demo-bucket',
Key: 'HappyFace.jpg' }
2021-02-18T21:41:00.215Z    12b3cae7-5f4e-415e-93e6-416b8f8b66e6
INFO    CONTENT TYPE: image/jpeg
END RequestId: 12b3cae7-5f4e-415e-93e6-416b8f8b66e6
REPORT RequestId: 12b3cae7-5f4e-415e-93e6-416b8f8b66e6
Duration: 976.25 ms    Billed Duration: 977 ms    Memory Size:
128 MB    Max Memory Used: 90 MB    Init Duration: 430.47 ms

Request ID
12b3cae7-5f4e-415e-93e6-416b8f8b66e6
```

Test the Lambda function with the Amazon S3 trigger



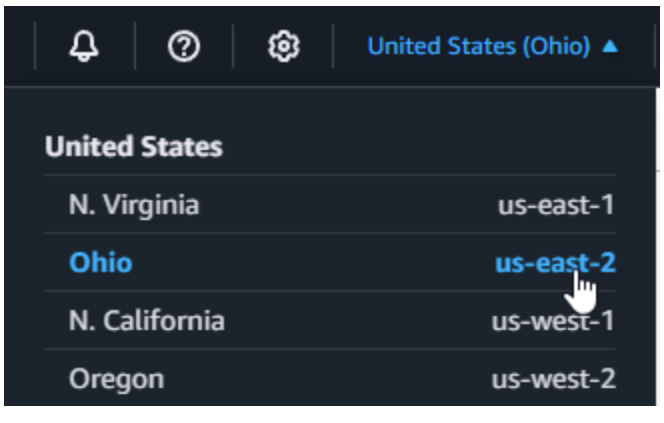
To test your function with the configured trigger, upload an object to your Amazon S3 bucket using the console. To verify that your Lambda function ran as expected, use CloudWatch Logs to view your function's output.

To upload an object to your Amazon S3 bucket

1. Open the Buckets page of the Amazon S3 console and choose the bucket that you created earlier.
2. Choose **Upload**.
3. Choose **Add files** and use the file selector to choose an object you want to upload. This object can be any file you choose.
4. Choose **Open**, then choose **Upload**.

To verify the function invocation using CloudWatch Logs

1. Open the CloudWatch console.
2. Make sure you're working in the same AWS Region you created your Lambda function in. You can change your Region using the drop-down list at the top of the screen.



3. Choose **Logs**, then choose **Log groups**.
4. Choose the log group for your function (`/aws/lambda/s3-trigger-tutorial`).
5. Under **Log streams**, choose the most recent log stream.
6. If your function was invoked correctly in response to your Amazon S3 trigger, you'll see output similar to the following. The `CONTENT TYPE` you see depends on the type of file you uploaded to your bucket.

```
2022-05-09T23:17:28.702Z        0cae7f5a-b0af-4c73-8563-
a3430333cc10    INFO    CONTENT TYPE: image/jpeg
```

Clean up your resources

You can now delete the resources that you created for this tutorial, unless you want to retain them. By deleting AWS resources that you're no longer using, you prevent unnecessary charges to your AWS account.

To delete the Lambda function

1. Open the Functions page of the Lambda console.
2. Select the function that you created.
3. Choose **Actions**, **Delete**.
4. Type `confirm` in the text input field and choose **Delete**.

To delete the execution role

1. Open the Roles page of the IAM console.
2. Select the execution role that you created.
3. Choose **Delete**.
4. Enter the name of the role in the text input field and choose **Delete**.

To delete the S3 bucket

1. Open the Amazon S3 console.
2. Select the bucket you created.
3. Choose **Delete**.
4. Enter the name of the bucket in the text input field.
5. Choose **Delete bucket**.

Next steps

In Tutorial: Using an Amazon S3 trigger to create thumbnail images, the Amazon S3 trigger invokes a function that creates a thumbnail image for each image file that is uploaded to a bucket. This tutorial requires a moderate level of AWS and Lambda domain knowledge. It demonstrates how to create resources using the AWS Command Line Interface (AWS CLI) and how to create a .zip file archive deployment package for the function and its dependencies.